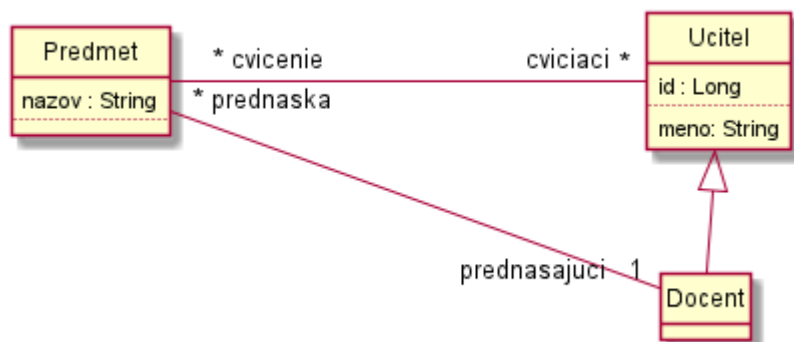


Implementačná úloha 1.

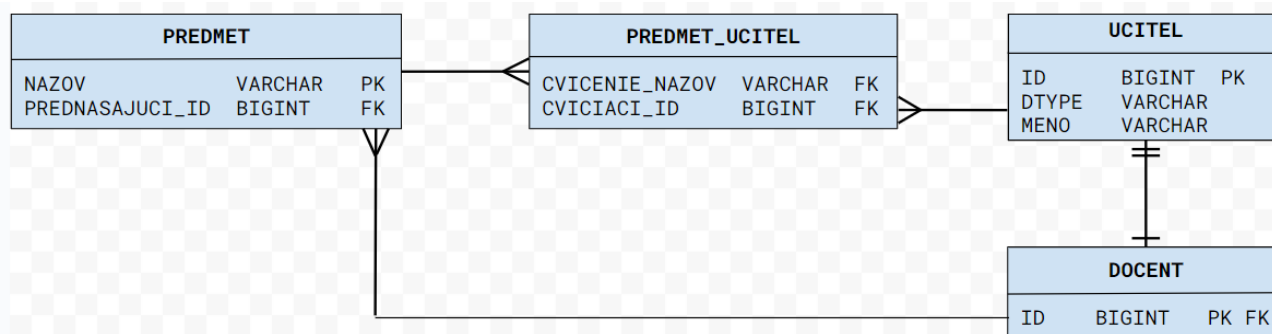
[20 b]

Adresár **dbapp1ta** je maven projekt pre vývoj databázovej aplikácie. V balíku **dbapp** sú vytvorené entitné triedy podľa UML diagramu tried, zatiaľ bez JPA anotácií:



Úlohy

A) Triedy v balíku dbapp **doplňte o JPA anotácie** tak, aby z nich vygenerovaná databáza mala štruktúru znázornenú na relačnom diagrame:



Dôležité je aby vygenerované tabuľky mali **presne tú istú štruktúru** ako je uvedená v **relačnom diagrame** hore. (Názvy tabuliek, stĺpcov a ich typy musia byť zachované, na poradí stĺpcov nezáleží).

Stĺpec MENO v tabuľke UCITEL bude mať **povinné** a **unikátne** hodnoty.

Kľúčový stĺpec ID môže (ale nemusí) byť autogenerovaný.

B) V balíku dbapp sa okrem entitných tried nachádza aj súbor **DbApp.java** obsahujúci deklaráciu metódy:

`Predmet novýPredmet( EntityManager em, String nazov, String meno ) throws Exception`

Metóda slúži na vytvorenie nového predmetu v databáze, pričom dostane argumenty: manažér entít, názov predmetu a meno osoby, ktorá bude jeho prednášajúcim aj cvičiacim.

Vašou úlohou je implementácia metódy novýPredmet podľa nasledujúcej špecifikácie:

- Metóda najprv preverí, či predmet so zadaným názvom už v databáze neexistuje.
 - o Ak nie je zadaný názov predmetu, metóda neurobí nič, len vyhodí výnimku so správou *chyba*.
 - o Ak predmet už existuje, metóda nerobí žiadne zmeny v databáze, len vyhodí výnimku so správou *duplicita*.
- Ak predmet s daným názvom neexistuje, vytvorí ho.
- Ak je meno osoby zadané, vytvorí nového profesora s daným menom a urobí ho prednášajúcim aj cvičiacim novovytvoreného predmetu. (Ak nie je zadané, nový predmet ostane zatiaľ bez prednášajúceho aj cvičiacich.)
- Metóda vráti objekt novovytvoreného predmetu.

Poznámky: Okrem tejto metódy obsahuje súbor aj **main** funkciu. Funkcia main je už implementovaná. Slúži len pre vás na testovanie. Jej implementáciu si môžete zmeniť podľa potreby. Nevyhodnocuje sa. Projekt už obsahuje súbor **persistence.xml** s konfiguráciou pripojenia na databázu postgresql.

Odovzdajte balík **dbapp** zozipovaný ako súbor **dbapp.zip**

Implementačná úloha 2.

[20 b]

V adresári **restapp** nájdete maven projekt pre vývoj webovej aplikácie s REST servisom.

Projekt obsahuje:

- súbor **pom.xml** so všetkými závislosťami potrebnými pre vývoj servisu. Nie je ho potrebné dopĺňať.
- balík **rest**. **V tomto balíku budete vytvárať všetky triedy** (triedu implementujúcu servis aj dátovú/é triedu)

V balíku rest sa už nachádza konfiguračná trieda REST aplikácie JAXRSConfiguration. Netreba ju meniť.

Špecifikácia servisu

Servis bude poskytovať informácie o jedlách v ponuke našej jedálne pre jednotlivé dni:

Koreňový resource nazvite **ponuka**.

Ponuka na konkrétny deň bude prístupná ako resource s URL **ponuka/{datum}**

kde **datum** je identifikátor dňa.

Jednotlivé jedlá v ponuke dňa budú prístupné ako subresoursy s URL **ponuka/{datum}/{n}**

kde **n** je poradové číslo jedla v ponuke daného dňa, pričom jedlá **sú číslované za sebou od 1**.

Pre URL **ponuka/{datum}** implementujte metódy:

- **GET** pre MIME text/plain: vráti aktuálny počet jedál v ponuke pre daný deň.
Ak pre daný deň neexistuje ponuka, vráti 0.
- **Post** pre MIME application/xml: Pridá nové jedlo do ponuky daného dňa.
Akceptuje xml element **jedlo** so štruktúrou:

```
<jedlo>
  <cena>3.5</cena>
  <nazov>gulas</nazov>
</jedlo >
```

Metóda vráti poradové číslo nového jedla v ponuke (MIME text/plain).

Pozor. V ponuke dňa by nemali byť dve jedlá s rovnakým názvom, preto by servis mal najprv skontrolovať, či jedlo s rovnakým názvom už v ponuke dňa nie je. Ak tam už je, nové jedlo nepridá ale vráti poradové číslo existujúceho jedla.

Ak ponuka pre daný deň ešte neexistuje (neobsahuje žiadne jedlá), inicializuje ju.

Pre URL **ponuka/{datum}/{n}** implementujte metódy:

- **GET** pre MIME application/xml: Vráti informácie o jedle s poradovým číslom **n** ako xml s horeuvedenou štruktúrou.
- **DELETE**: Odstráni jedlo s poradovým číslom **n** z ponuky na daný **deň**.
Po odstránení jedla z ponuky sa poradové čísla jedál za ním znížia.

Implementácia servisu

Všetky dáta budú udržiavané **len v pamäti servisu**. Vytvorte si pre dáta vhodné dátové štruktúry a (ak treba) dátové triedy. Použite anotáciu **@javax.inject.Singleton**. Nezabudnite tiež na anotáciu **@XmlRootElement**.

Dôležité - inicializácia servisu: Pri štarte si aplikácia inicializuje ponuku na dnešný deň, teda s dátumom **20-5-2024**, ktorá bude obsahovať jediné jedlo s názvom **gulas** a cenou **3.5**.

Poznámky:

- Uvedené xml je len príklad, ktorý ilustruje požadovanú štruktúru. XML dokumenty, ktorými pracuje servis, majú elementy s rovnakými názvami a štruktúrou, odlišovať môžu len textovým obsahom. Poradie elementov nie je dôležité.
- Ak klient volá metódy pre neexistujúci resource, server nesmie vrátiť kód HTTP 50*

Pokyny pre odovzdanie

Odovzdajte balík **rest** zozipovaný ako súbor **rest.zip**